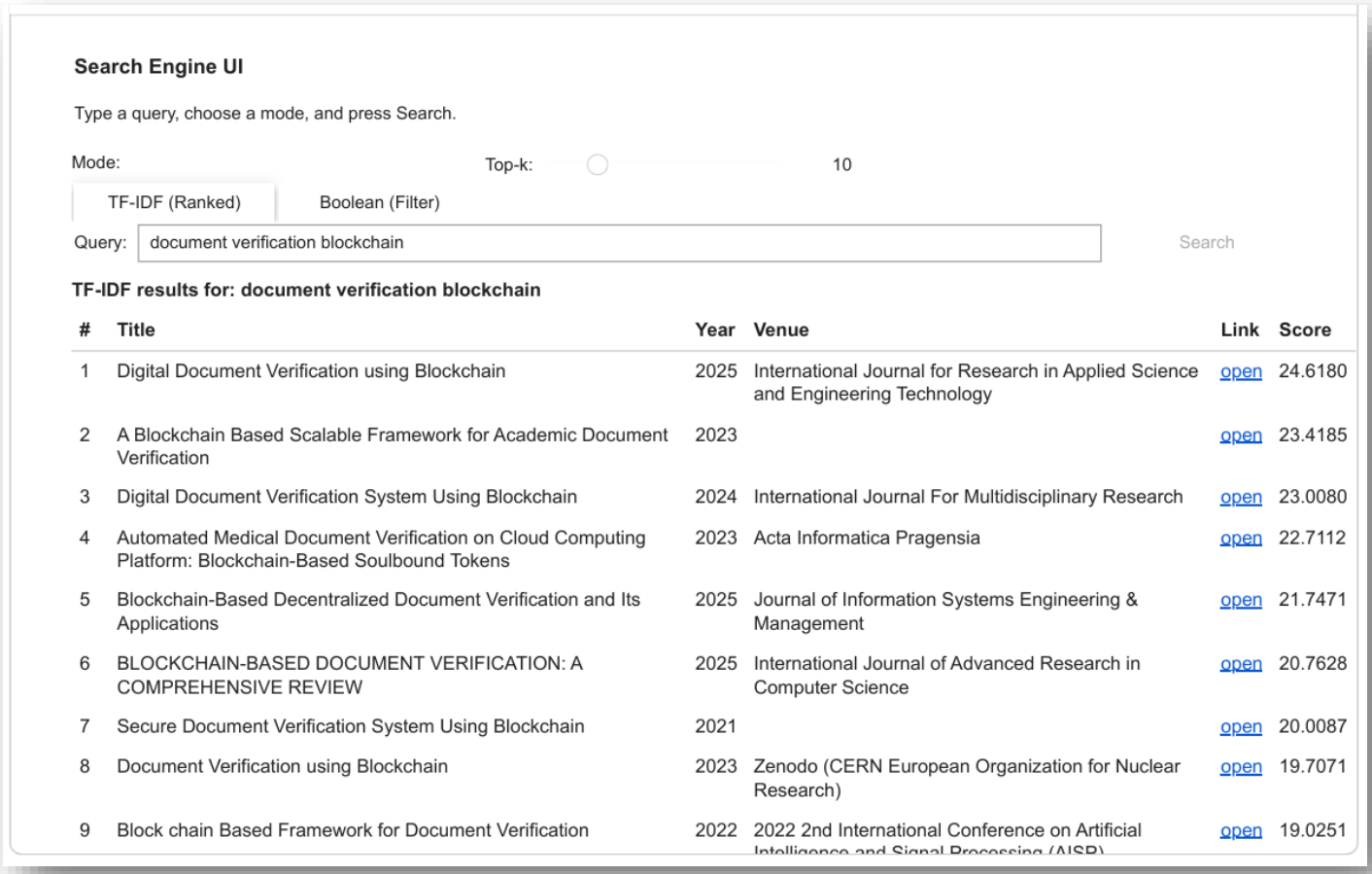


Search Engine UI

Type a query, choose a mode, and press Search.

Mode: Top-k: 10

TF-IDF (Ranked) Boolean (Filter)

Query: document verification blockchain Search

TF-IDF results for: document verification blockchain

#	Title	Year	Venue	Link	Score
1	Digital Document Verification using Blockchain	2025	International Journal for Research in Applied Science and Engineering Technology	open	24.6180
2	A Blockchain Based Scalable Framework for Academic Document Verification	2023		open	23.4185
3	Digital Document Verification System Using Blockchain	2024	International Journal For Multidisciplinary Research	open	23.0080
4	Automated Medical Document Verification on Cloud Computing Platform: Blockchain-Based Soulbound Tokens	2023	Acta Informatica Pragensia	open	22.7112
5	Blockchain-Based Decentralized Document Verification and Its Applications	2025	Journal of Information Systems Engineering & Management	open	21.7471
6	BLOCKCHAIN-BASED DOCUMENT VERIFICATION: A COMPREHENSIVE REVIEW	2025	International Journal of Advanced Research in Computer Science	open	20.7628
7	Secure Document Verification System Using Blockchain	2021		open	20.0087
8	Document Verification using Blockchain	2023	Zenodo (CERN European Organization for Nuclear Research)	open	19.7071
9	Block chain Based Framework for Document Verification	2022	2022 2nd International Conference on Artificial Intelligence and Signal Processing (AISP)	open	19.0251

IR PROJECT

Aitana Cantó Rico

Building a Search Engine for Academic Papers

Collab link

Index

1. Introduction and Motivation 2
2. Challenges and Design Decisions 2
3. Data Collection 3
4. Text Preprocessing 3
5. Index Construction 4
6. Boolean Retrieval 4
   6.1 Motivation 4
   6.2 Supported Operators and Query Syntax 5
   6.3 Query Parsing and Evaluation 5
   6.4 Boolean Retrieval Implementation 5
   6.5 Result Presentation 6
   6.6 Experimental Examples 6
   6.7 Strengths and Limitations 6
7. Ranked Retrieval using TF-IDF 6
   7.1 Motivation 6
   7.2 Query Processing 6
   7.3 TF-IDF Weighting Scheme 7
   7.4 Ranking Algorithm 7
   7.5 Result Presentation 7
   7.6 Experimental Results 7
   7.7 Discussion 8
8. Evaluation 8
   8.1 Evaluation Setup 8
   8.2 Manual Relevance Judgments (qrels) 9
   8.3 Evaluation Metrics 9
   8.4 Results Summary 10
   8.5 Discussion 10
9. Results Analysis (Per-Query Interpretation) 10
   9.1 Queries with Excellent Performance (Precision@10 = 1.0) 10
   9.2 Queries with Strong but Not Perfect Performance (Precision@10 ≈ 0.7–0.8) 11
   9.3 Queries with Moderate Precision (Precision@10 ≈ 0.4–0.6) 12
   9.4 Queries with Low Precision (Precision@10 ≤ 0.3) 13
   9.5 Overall Interpretation and What This Shows 13
   9.6 Potential Improvements 14
   10. Final Conclusions 14

1. Introduction and Motivation

The goal of this project was to design and implement a small academic search engine focused on **document verification, authenticity, and the reliability of information**, with a particular emphasis on AI-generated text and large language models. The system covers the complete information retrieval pipeline, from data collection and preprocessing to indexing, querying, ranking, and evaluation.

I chose this topic deliberately because it is closely related to my academic interests and future plans. In the next semester, I will start my Bachelor's Thesis (TFG), which will focus on a topic related to document validation. For this reason, I wanted this project to serve not only as a course assignment, but also as a **practical foundation** for the concepts, tools, and challenges that I will face in my final degree project.

2. Challenges and Design Decisions

During the development of the project, I encountered several practical challenges that influenced the final design of the system.

One of the first issues appeared during data collection. OpenAlex provides a lot of useful information, but abstracts are returned in an inverted index format instead of plain text. To solve this, I implemented a reconstruction step to convert the abstract into readable text before storing and processing it. Without this step, the documents could not be properly indexed.

Another challenge was avoiding duplicate documents. Since I used multiple keywords to build the dataset, the same paper could appear in different queries. To handle this, I kept a global set of document IDs and ensured that each document was only added once to the collection.

Text preprocessing also required several decisions. I chose to keep preprocessing relatively simple: lowercasing, stopword removal, and token filtering. I decided not to use stemming or lemmatization, because I wanted the system to remain easy to understand and to avoid introducing additional sources of error.

From a retrieval point of view, one of the main challenges was query ambiguity. Some terms, such as verification or authenticity, are used in many different research areas. As a result, the system sometimes retrieves documents that share vocabulary with the query but are not fully relevant. This limitation comes from using TF-IDF, which is based on term overlap and does not capture deeper semantic meaning.

Finally, the evaluation process was also a challenge. Relevance judgments were done manually and only for the top-10 results of each query. This was a deliberate decision to keep the evaluation feasible while still allowing meaningful analysis of ranking quality.

3. Data Collection

To build the document collection, I created a domain-specific academic corpus using the OpenAlex API, which provides open access to scholarly metadata from a wide range of research fields. The goal of this step was to collect papers that are directly related to the main topics of the project: document verification and authenticity, information trustworthiness, misinformation detection, and the reliability of AI-generated text.

Rather than using a single broad query, I defined a set of focused keywords covering different aspects of the problem domain. This allowed me to capture a more diverse but still relevant set of documents. The keywords used were:

- document verification
- document validation
- document authenticity
- fact checking document text
- information credibility trustworthiness
- LLM hallucination detection
- AI-generated text reliability trust verification
- misinformation detection text document
- consistency checking generated text LLM

Each keyword was submitted to the OpenAlex search endpoint using cursor-based pagination. This strategy made it possible to retrieve large result sets efficiently while respecting the API rate limits. Because the same document could appear in multiple queries, I kept a global set of document identifiers to ensure that each document was stored only once.

One practical issue I encountered was that OpenAlex returns abstracts in an inverted index format instead of plain text. To handle this, I implemented a reconstruction step that converts the inverted index into readable abstract text before saving the document.

For each document, I stored a unique identifier, title, reconstructed abstract, publication year, venue, URL (or DOI when available), citation count, and the keyword that triggered its retrieval. This information was preserved to support later preprocessing, indexing, ranking, and result presentation.

In total, **630 unique documents** were collected and saved in JSON Lines format (*raw_papers.jsonl*). This dataset forms the raw corpus used in the following stages of the system.



4. Text Preprocessing

Before building the index, I applied a preprocessing pipeline to clean and standardize the document texts. Since the documents were collected from different sources, their titles and abstracts contained formatting inconsistencies that needed to be handled before indexing.

For each document, I cleaned the title and abstract by removing special characters, soft hyphens, and line breaks. This step prevents formatting artifacts from being treated as meaningful terms during retrieval.

The cleaned text was then tokenized using a regular expression that extracts alphabetic words only. All tokens were converted to lowercase to ensure consistency. I also removed stopwords (such as *the*, *and*, *of*) because they do not add semantic value for retrieval. Tokens shorter than two characters were discarded, as they usually represent noise.

To keep the collection clean, I removed documents with very little textual content. Any document with fewer than five tokens after preprocessing was excluded, since such documents are unlikely to be useful for retrieval.

After preprocessing, the final dataset contained **615 documents**. These were stored in *processed_papers.jsonl*, including the cleaned text, token list, token count, and metadata. This processed collection was used as the input for index construction and all retrieval experiments.

```
Input docs: 630
Processed docs: 615
Skipped docs: 15
Saved to: data/processed/processed_papers.jsonl
```

5. Index Construction

After preprocessing the documents, I built an inverted index to support fast and efficient retrieval. For each document, I computed term frequencies from the tokenized text and stored them in postings lists mapping terms to document IDs and their frequencies.

```
for doc in iter_jsonl(IN_PATH):
    doc_id = doc["doc_id"]
    tokens = doc["tokens"]
    n_docs += 1

    # doc length
    doc_lengths[doc_id] = int(doc.get("len_tokens", len(tokens)))

    # metadata
    doc_meta[doc_id] = {
        "title": doc.get("title", ""),
        "year": doc.get("year", None),
        "venue": doc.get("venue", None),
        "url": doc.get("url", None),
        "citationCount": doc.get("citationCount", 0),
    }
```

Alongside the inverted index, I created additional data structures including a vocabulary with document frequencies, a table with document lengths, and a metadata dictionary containing titles, years, venues, URLs, and citation counts. These structures are required for TF-IDF ranking and result presentation.

All index components were saved using pickle files so they can be loaded quickly during querying and evaluation. The final index contains 615 documents and 9,732 unique terms.

```
Indexed documents: 615
Vocabulary size (unique terms): 9732
Saved postings to: data/index/postings.pkl
Saved vocab(df) to: data/index/vocab.pkl
Saved doc lengths to: data/index/doc_lengths.pkl
Saved doc meta to: data/index/doc_meta.pkl
```

6. Boolean Retrieval

6.1 Motivation

I implemented Boolean retrieval to allow precise and easily interpretable filtering of documents using explicit logical conditions. Unlike ranked retrieval models, Boolean search only returns documents that strictly satisfy the query constraints. This makes it especially useful as a first filtering step to reduce the size of the result set before applying a ranking model.

In this project, Boolean retrieval is mainly used to explore the collection and to verify that the indexing process works correctly.

6.2 Supported Operators and Query Syntax

The Boolean retrieval module supports the following operators:

- **AND**
- **OR**
- **NOT**
- **Parentheses** for grouping and precedence control

This allows users to build expressive queries that combine multiple concepts or explicitly exclude unwanted topics. Some example queries tested in the system are:

- document AND verification
- (llm OR ai) AND NOT misinformation
- fact AND checking AND (trust OR credibility)

6.3 Query Parsing and Evaluation

Boolean queries are processed in three main steps:

1. **Tokenization and normalization**, where query terms are lowercased and Boolean operators are preserved.
2. **Conversion to Reverse Polish Notation (RPN)** using the shunting-yard algorithm, which ensures correct operator precedence (NOT > AND > OR).
3. **Evaluation of the RPN expression** using a stack-based approach, where each term is mapped to a set of document identifiers obtained from the inverted index.

This approach guarantees a clear and unambiguous interpretation of Boolean expressions and allows efficient evaluation using set operations.

6.4 Boolean Retrieval Implementation

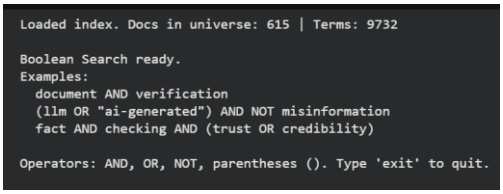
For each query term, the system retrieves the set of documents in which the term appears. Boolean operators are implemented using standard set operations:

- **AND** → set intersection
- **OR** → set union
- **NOT** → set complement with respect to the document universe

The document universe corresponds to the full set of indexed documents. This design makes the implementation both simple and efficient, even for more complex Boolean queries.

6.5 Result Presentation

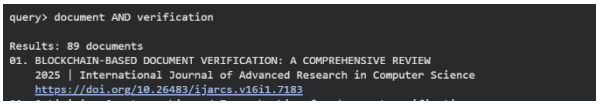
Boolean retrieval returns an unranked set of documents that satisfy the query constraints. For better readability, the results are displayed ordered by publication year (descending) and citation count. Each result is shown together with its metadata, including title, publication year, venue, and URL.



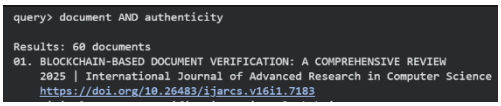
6.6 Experimental Examples

To validate the implementation, several Boolean queries were executed:

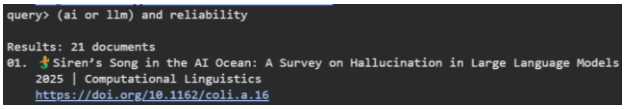
- The query document AND verification retrieved **89 documents** related to document verification systems and techniques.



- The query document AND authenticity returned **60 documents** focusing on document authenticity and validation.



- More restrictive queries such as (trust OR trustworthiness) AND document produced smaller and more focused result sets.



These experiments confirm that the Boolean retrieval component correctly applies logical constraints and filters the collection as expected.

6.7 Strengths and Limitations

Boolean retrieval offers full user control and high precision with respect to explicit query conditions. However, it does not provide any relevance ranking and may return very large or very small result sets depending on how specific the query is. For this reason, in this system Boolean retrieval is mainly used as a filtering and exploration tool, and it is complemented by the ranked retrieval model described in the next section.

7. Ranked Retrieval using TF-IDF

7.1 Motivation

Although Boolean retrieval is useful for filtering documents using strict conditions, it does not provide any way to order results by relevance. To overcome this limitation, I implemented a ranked retrieval model based on TF-IDF. This allows the system to score and rank documents according to how relevant they are to a given query, which is essential for a practical search engine.

7.2 Query Processing

Queries are processed using the same normalization strategy applied during document preprocessing. Each query is lowercased, tokenized using a word-based tokenizer, and filtered by removing stopwords and very short tokens. This ensures consistency between query terms and the indexed document terms, avoiding mismatches caused by formatting or capitalization differences.

7.3 TF-IDF Weighting Scheme

The ranking model uses a standard **TF-IDF weighting scheme** defined as follows:

- **Term Frequency (TF)** is log-scaled to reduce the impact of very frequent terms.

$$TF(t,d) = 1 + \log\left(tf_{t,d}\right)$$

- **Inverse Document Frequency (IDF)** is smoothed to avoid division by zero and negative values.

$$IDF(t) = \log\left(\frac{N+1}{df_t+1}\right) + 1$$

- **Document Score** is computed as the sum of TF-IDF weights over all query terms.

$$Score(d,q) = \sum_{t \in q} TF(t,d) \times IDF(t)$$

This formulation favors documents that contain query terms frequently while penalizing terms that appear in many documents.

7.4 Ranking Algorithm

For each query term that appears in the vocabulary, the system retrieves its posting list from the inverted index. Each document in the posting list receives a partial score based on the TF-IDF weight of the term. These partial scores are accumulated across all query terms to compute the final document score.

Documents are then sorted in descending order of their scores, producing a ranked list of results.

7.5 Result Presentation

The ranked retrieval module returns the top-*k* documents for a given query. Each result is displayed together with:

- TF-IDF relevance score
- Title
- Publication year
- Venue
- URL (when available)

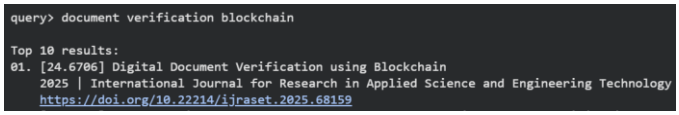


This presentation makes it easy to evaluate both the relevance and the academic context of each retrieved document.

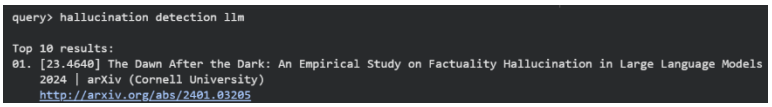
7.6 Experimental Results

I tested the TF-IDF ranking model using several queries related to the project domain. For example:

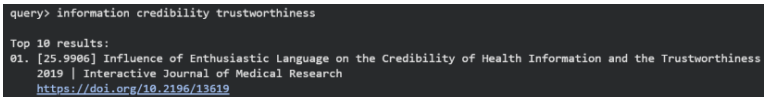
- The query *"document verification blockchain"* returned highly relevant papers focused on blockchain-based document verification systems.



- The query *"hallucination detection llm"* retrieved recent and influential works addressing hallucinations in large language models.



- Queries related to *information credibility* and *trustworthiness* successfully ranked documents from information science and communication research.



These experiments show that the TF-IDF model can prioritize documents that are clearly aligned with the user's information need.

7.7 Discussion

TF-IDF provides a strong and interpretable baseline for ranked retrieval. It is computationally efficient, simple to implement, and works well for a medium-sized collection like the one used in this project. However, it does not model term dependencies or semantic relationships between words, and it only partially accounts for document length effects. More advanced ranking approaches (such as BM25 or neural models) could be explored as future improvements.

8. Evaluation

8.1 Evaluation Setup

To evaluate the quality of the search engine, I designed a small evaluation framework based on manually annotated relevance judgments (qrels). The goal of this evaluation was not to create a large benchmark, but to check whether the system behaves reasonably for queries that are directly related to the project motivation.

I evaluated the system using **12 queries**, all of them aligned with the main topics of the project: document verification and authenticity, trustworthiness of information, claim verification, hallucination detection in LLMs, groundedness, and AI-generated misinformation.

The evaluation setup is defined as follows:

- **Top-k cutoff:** $k = 10$
- **Ranking method:** TF-IDF (the same model used in the ranked retrieval module)
- **Relevance labels:** Binary (relevant = 1, non-relevant = 0)
- **Storage:** relevance judgments are saved in data/eval/qrels.json

During development, I included an option to reset previous judgments using RESET_QRELS = True, which allows running a clean evaluation without mixing results from earlier annotation sessions.

8.2 Manual Relevance Judgments (qrels)

For each query, the system retrieves the top 10 documents ranked by TF-IDF. I then manually inspected these results and marked which documents were relevant to the query by entering the corresponding ranks (for example: 1 2 5).

For every query, the evaluation script stores:

- the list of retrieved document IDs
- a binary relevance vector of length 10

This creates a lightweight but reproducible evaluation process, since all judgments are saved to disk and reused when computing the final metrics.

8.3 Evaluation Metrics

The following standard Information Retrieval metrics are computed at $k = 10$:

- **Precision@10**, which measures how many of the top-10 retrieved documents are relevant:

$$Precision@10 = \frac{\#\text{relevant in top-10}}{10}$$

- **Recall@10**, which measures how many of the judged relevant documents appear in the top-10 list:

$$Recall@10 = \frac{\#\text{relevant in top-10}}{\#\text{relevant in judged set}}$$

Since relevance is only judged for the retrieved top-10 documents, recall here reflects whether the system retrieves all relevant items within that judged set.

- **F1@10**, the harmonic mean of precision and recall:

$$F1@10 = \frac{2PR}{P + R}$$

- **Average Precision (AP@10)**, which evaluates ranking quality by rewarding relevant documents that appear earlier in the ranked list:

$$AP@10 = \frac{1}{R}\sum_{i=1}^{10} Precision@i \cdot rel_i$$

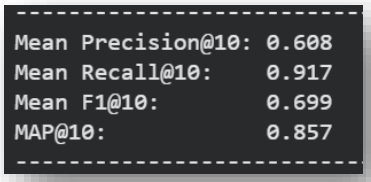
- **Mean Average Precision (MAP@10)**, computed as the mean of AP@10 across all queries:

$$MAP@10 = \frac{1}{|Q|}\sum_{q \in Q} AP@10(q)$$

8.4 Results Summary

The evaluation produces both per-query metrics and overall averages. Using $k = 10$, the final aggregated results are:

- **Mean Precision@10:** 0.608
- **Mean Recall@10:** 0.917
- **Mean F1@10:** 0.699
- **MAP@10:** 0.857



These results show that the system retrieves a large proportion of relevant documents for most queries (high recall), while precision varies depending on how specific or ambiguous the query terms are.

8.5 Discussion

The evaluation confirms that TF-IDF ranking works particularly well for queries that closely match the dataset focus, such as *document verification* or *LLM hallucination detection*. For these queries, both precision and recall are very high.

Lower precision values are observed for broader or more ambiguous queries, such as *ai generated misinformation detection*, where retrieved documents may be related to AI or misinformation in general but do not always match the exact intent of the query. This behavior is expected for a purely lexical ranking model like TF-IDF.

Overall, the evaluation demonstrates that the indexing and ranking pipeline is correctly implemented and produces meaningful ranked results. At the same time, it highlights natural limitations of TF-IDF and points to potential improvements, such as better query formulation, phrase handling, or the use of alternative ranking models like BM25.

9. Results Analysis (Per-Query Interpretation)

In this section, I analyse the evaluation results obtained at $k = 10$and explain how the behaviour of my search engine changes depending on the type of query. The goal is to understand **why some queries perform very well** with TF-IDF, while others retrieve more partially related documents.

9.1 Queries with Excellent Performance (Precision@10 = 1.0)

Some queries achieved perfect precision and recall. In these cases, the query terms are very specific and closely match the vocabulary used in the document titles and abstracts stored in the index. As a result, TF-IDF is able to rank relevant documents very accurately.

- **document verification**
  Precision@10 = 1.000, AP@10 = 1.000
  Many documents in the collection explicitly use the term *document verification*, especially in the context of blockchain-based and security-oriented systems. Since this phrase appears frequently and consistently, almost all retrieved documents are relevant.

```
Query: document verification
  Relevant in top-10: 10/10
  Precision@10: 1.000
  Recall@10:    1.000
  F1@10:        1.000
  AP@10:        1.000
```

- **identity document verification**
  Precision@10 = 1.000, AP@10 = 1.000
  This is an even more specific query. Most retrieved papers focus directly on identity documents, datasets, or KYC verification systems, which matches the intended meaning very well.

```
Query: identity document verification
  Relevant in top-10: 10/10
  Precision@10: 1.000
  Recall@10:    1.000
  F1@10:        1.000
  AP@10:        1.000
```

- **llm hallucination detection**
  Precision@10 = 1.000, AP@10 = 1.000
  The dataset contains many recent papers that explicitly mention hallucinations, detection methods, and LLMs. The strong overlap between query terms and document vocabulary leads to excellent ranking performance.

```
Query: llm hallucination detection
  Relevant in top-10: 10/10
  Precision@10: 1.000
  Recall@10:    1.000
  F1@10:        1.000
  AP@10:        1.000
```

In these cases, TF-IDF works very well because the query terms are distinctive and strongly correlated with the actual topic of the documents.

9.2 Queries with Strong but Not Perfect Performance (Precision@10 ≈ 0.7–0.8)

Other queries still perform well, but with slightly lower precision. This usually happens when one or more query terms are used in multiple research contexts.

- **document authenticity**
  Precision@10 = 0.800, AP@10 = 0.907
  While many retrieved documents are about technical document authenticity and verification, the term *authenticity* also appears in legal, philosophical, or historical studies. This introduces some non-technical results into the top-10 list.

```
Query: document authenticity
  Relevant in top-10: 8/10
  Precision@10: 0.800
  Recall@10:    1.000
  F1@10:        0.889
  AP@10:        0.907
```

- **groundedness retrieval augmented generation**
  Precision@10 = 0.700, AP@10 = 0.744
  Although groundedness and RAG are relevant to hallucination mitigation, some retrieved papers focus more broadly on AI reliability or RAG architectures without explicitly addressing hallucinations or verification.

```
Query: groundedness retrieval augmented generation
  Relevant in top-10: 7/10
  Precision@10: 0.700
  Recall@10:    1.000
  F1@10:        0.824
  AP@10:        0.744
```

- **factuality evaluation llm**
  Precision@10 = 0.800, AP@10 = 0.735
  The term *factuality* is used across different evaluation settings, including general LLM evaluation and agent-based systems. This explains why not all retrieved documents are strictly about factuality evaluation methods.

```
Query: factuality evaluation llm
  Relevant in top-10: 8/10
  Precision@10: 0.800
  Recall@10:    1.000
  F1@10:        0.889
  AP@10:        0.735
```

These results show that TF-IDF still performs well, but broader terminology increases the chance of partially related matches.

9.3 Queries with Moderate Precision (Precision@10 ≈ 0.4–0.6)

Some queries retrieve a mix of highly relevant and loosely related documents. In these cases, the wording is relevant to the project but not specific enough to fully constrain the results.

- **fact checking claim verification**
  Precision@10 = 0.600, AP@10 = 0.976
  Relevant documents are usually ranked very high, which explains the strong AP value. However, the query also retrieves papers about fact-checking in journalism or misinformation studies, not only claim verification in an NLP or IR context.

```
Query: fact checking claim verification
  Relevant in top-10: 6/10
  Precision@10: 0.600
  Recall@10:    1.000
  F1@10:        0.750
  AP@10:        0.976
```

- **rag hallucination detection**
  Precision@10 = 0.400, AP@10 = 0.917
  Most of the top-ranked results are relevant, but lower-ranked documents include general hallucination detection papers that are not explicitly focused on RAG-based approaches.

```
Query: rag hallucination detection
  Relevant in top-10: 4/10
  Precision@10: 0.400
  Recall@10:    1.000
  F1@10:        0.571
  AP@10:        0.917
```

- **signature verification**
  Precision@10 = 0.500, AP@10 = 1.000
  This query retrieves several highly relevant documents related to digital signatures and document authenticity, which are ranked very early in the list. However, it also retrieves papers from broader security or cryptography domains where signature verification is discussed in a more general sense. This leads to moderate precision, despite excellent ranking quality at the top of the results.

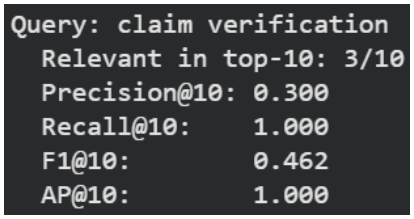
```
Query: signature verification
  Relevant in top-10: 5/10
  Precision@10: 0.500
  Recall@10:    1.000
  F1@10:        0.667
  AP@10:        1.000
```

Even when precision is moderate, the high AP values indicate that the most relevant documents tend to appear early in the ranking, which is desirable in practice.

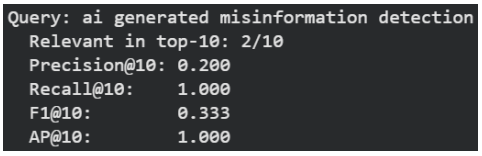
9.4 Queries with Low Precision (Precision@10 ≤ 0.3)

The lowest precision values appear for queries that are highly ambiguous or use very generic terms.

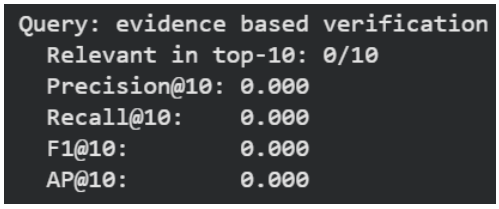
- **claim verification**
  Precision@10 = 0.300, AP@10 = 1.000
  Although relevant documents are ranked early, the term *verification* appears in many unrelated contexts (document verification, system verification, security protocols), which introduces noise.



- **ai generated misinformation detection**
  Precision@10 = 0.200, AP@10 = 1.000
  This query combines broad concepts (*AI*, *misinformation*), so many retrieved documents discuss these topics without focusing specifically on detection methods.



- **evidence based verification**
  Precision@10 = 0.000, AP@10 = 0.000
  This query is particularly ambiguous. The phrase *evidence-based* is widely used in medicine and education, and *verification* appears in multiple technical domains. As a result, lexical overlap alone is not sufficient to retrieve relevant documents.



These cases clearly show the limitations of a purely term-based ranking model.

9.5 Overall Interpretation and What This Shows

Overall, the results reflect realistic behaviour for a TF-IDF-based search engine:

- The system performs extremely well when queries contain **specific, domain-aligned terms**.
- Performance degrades when queries include **generic or ambiguous vocabulary**.
- In many cases, even when precision is low, **Average Precision remains high**, meaning that relevant documents are still ranked near the top.

This confirms that the implemented indexing and ranking pipeline works correctly and produces meaningful results for its intended scope.

9.6 Potential Improvements

Based on these observations, several improvements could be explored in future work:

- Phrase-based retrieval (e.g., treating *claim verification* as a single unit)
- Query expansion using synonyms or related terms
- More advanced ranking models such as **BM25**

10. Final Conclusions

In this project, I implemented a complete information retrieval system, covering data collection, preprocessing, indexing, retrieval, and evaluation. Working on all these stages helped me understand how search engines work beyond theory and how different design choices affect the final results.

I learned that classical models like Boolean retrieval and TF-IDF can work very well when queries are specific and well aligned with the dataset, but they also have clear limitations when queries are ambiguous or broad. The evaluation results reflect this behavior and helped me better understand the strengths and weaknesses of lexical retrieval models.

Overall, this project makes sense as a preparation for my future Bachelor's Thesis, since it is directly related to document validation and verification. It also gave me practical experience with real academic data and reinforced my understanding of core information retrieval concepts in a realistic scenario.